SIMATS ENGINEERING

ASSESSMENT TOOLS

COURSE NAME: Theory of Computation
COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20 **Total Marks: 20**

Duration : 1 Hour

Mock Interview question

Code of Conduct:

I **J.Goutham Kumar Reddy/ 192373041 (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

J.Goutham Kumar Reddy

SIMATS ENGINEERING

ASSESSMENT TOOLS

1. **What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?**
2. **Define the components of a PDA formally.**
3. **What is a stack in PDA? Why is it important?**
4. **What is an Instantaneous Description (ID) in PDA?**
5. **What are the different types of moves in a PDA?**
6. **What is acceptance by final state and empty stack?**
7. **Define the language accepted by a PDA.**
8. **How does a PDA process a string using IDs?**
9. **Differentiate between Deterministic PDA and Non-Deterministic PDA.**
10. **Why are PDAs equivalent to Context-Free Grammars (CFG)?**
11. **What types of languages are accepted by PDA? Give examples.**
12. **What is a Turing Machine? How is it more powerful than PDA?**
13. **Define the components of a Turing Machine.**
14. **Explain tape, head, and state transitions in a Turing Machine.**
15. **Explain programming techniques for Turing Machines.**
16. **What is storage in finite control in Turing Machines?**
17. **Compare FA, PDA, and TM based on memory, power, and language acceptance.**
18. **What is DFA?**
19. **Which language is accepted by a Turing Machine?**
20. **How are Turing Machines applied in real world applications?**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (5)	Level 3 (4)	Level 2 (2-3)	Level 1 (0-1)
Conceptual Understanding	30	Clear & complete understanding	Good understanding, minor gaps	Basic understanding, some misconceptions	Poor / incorrect understanding
Accuracy of Answer	25	Completely correct, no errors	Mostly correct, minor errors	Partially correct, noticeable errors	Incorrect / irrelevant
Application of Knowledge	20	Correctly applies concepts	Applies with minor mistakes	Limited / partially correct application	No / incorrect application
Completeness of Response	15	Covers all required parts	Covers most parts, minor omissions	Covers only some parts	Incomplete / missing
Clarity & Presentation	10	Clear, concise, well-organized	Understandable, minor issues	Somewhat unclear / poorly structured	Difficult to understand

ANSWERS

1.

A PDA is a finite automaton equipped with an additional auxiliary memory in the form of a stack (a LIFO data structure). It reads input symbols, changes state, and can push or pop symbols on the stack at each step.

Difference from FA: an FA has only a fixed, finite number of states and no external memory, so it can recognize only regular languages. A PDA's stack gives it unbounded memory, letting it recognize the larger class of context-free languages — e.g., a PDA can accept $a^n b^n$ (which requires counting/matching an unbounded number of a's against b's), but no FA can.

2.

A PDA is formally a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q0, Z0, F)$:

- Q — a finite set of states
- Σ — the finite input alphabet
- Γ — the finite stack alphabet
- δ — the transition function: $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow$ finite subsets of $Q \times \Gamma^*$
- $q0 \in Q$ — the initial state
- $Z0 \in \Gamma$ — the initial (bottom-of-stack) stack symbol
- $F \subseteq Q$ — the set of final/accepting states

3.

The stack is a Last-In-First-Out (LIFO) auxiliary storage attached to the PDA: symbols can only be pushed onto or popped from the top. It is important because it gives the PDA unbounded memory (unlike an FA's fixed states), allowing it to count and match nested or paired structures — such as balanced parentheses or equal numbers of a's and b's — which is exactly what distinguishes context-free languages from regular ones.

4.

An ID is a snapshot of the PDA's entire configuration at a given moment during a computation, written as a triple (q, w, γ), where q is the current state, w is the remaining unread portion of the input, and γ is the current stack content (top symbol listed first). A computation is a sequence of IDs connected by the $\vdash$ ("yields") relation, one per transition applied.

5.

- Read (input-consuming) move — the PDA reads one input symbol, changes state, and updates the stack based on $\delta(q, a, X)$.
- ε-move (input-independent move) — the PDA changes state and/or updates the stack without consuming any input symbol, based on $\delta(q, \varepsilon, X)$.
- Within either type, the stack operation itself can be a push (replace top with a longer string), a pop (replace top with ε), or a no-op replacement (replace top with a single symbol, leaving stack height unchanged).

6.

Acceptance by final state: the input string is accepted if, after the entire input is consumed, the PDA is in some state $q \in F$ (the stack content at that point does not matter).

Acceptance by empty stack: the input string is accepted if, after the entire input is consumed, the stack is completely empty (the current state does not matter, so F is typically irrelevant/not used).

These two acceptance modes are formally equivalent in power — any PDA using one mode can be mechanically converted to an equivalent PDA using the other, and both define exactly the class of context-free languages.

7.

For final-state acceptance: $L(M) = \{ w \in \Sigma^* \mid (q0, w, Z0) \vdash^* (q, \varepsilon, \gamma)$ for some $q \in F$ and some $\gamma \in \Gamma^* \}$.

For empty-stack acceptance: $N(M) = \{ w \in \Sigma^* \mid (q0, w, Z0) \vdash^* (q, \varepsilon, \varepsilon)$ for some $q \in Q \}$.

In words: the set of all input strings for which some sequence of valid moves takes the PDA from its initial configuration to an accepting configuration after consuming the whole string.

8.

Processing begins at the initial ID (q0, w, Z0), where w is the full input string. At each step, the PDA applies a transition matching the current state, the next unread input symbol (or ε), and the current stack-top symbol; this produces a new ID with the input shortened (if a symbol was read) and the stack updated per the transition's push/pop instruction. This repeats, forming a chain of IDs $(q0,w,Z0) \vdash (q1,w1,\gamma1) \vdash (q2,w2,\gamma2) \vdash \ldots$ until the input is exhausted; the string is accepted if the final ID satisfies the chosen acceptance condition (final state, or empty stack).

9.

Aspect	**DPDA**	**NPDA**
Transitions	At most one valid move for every (state, input symbol or ε, stack-top) combination	Can have multiple valid moves for the same combination
ε-move rule	If an ε-move is available on a stack symbol, no symbol-reading move may also be available on it (avoids ambiguity)	No such restriction — free mixing of choices
Language class	Deterministic Context-Free Languages (DCFL) — a proper subset of CFL	Full class of Context-Free Languages (CFL)
Closure under complement	Yes	No (in general)
Example it cannot handle	ww^R over general alphabets needs care; palindromes with no center marker are typically non-deterministic	Handles all CFLs, including ambiguous/backtracking cases

10.

PDAs and CFGs are equivalent because they generate/recognize exactly the same class of languages — the context-free languages — and there exist standard, mechanical, bidirectional constructions between them:

- CFG → PDA: for any CFG G, a PDA can be built that simulates G's leftmost derivations by pushing the start symbol and repeatedly replacing a variable on the stack top with the right-hand side of one of its productions, matching terminals against the input.
- PDA → CFG: for any PDA M, a CFG can be built whose variables A[p,q] represent 'strings that drive M from state p to state q while popping one stack symbol,' with productions derived directly from M's transition function.

Since a language is generated by some CFG if and only if it is accepted by some PDA, the two models are proven to have identical expressive power.

11.

PDAs accept exactly the Context-Free Languages (CFL) — the class that sits between regular languages and recursively enumerable languages in the Chomsky hierarchy.

- $L = \{a^n b^n \mid n \geq 1\}$ — equal counts of a's followed by b's
- Balanced parentheses / well-nested brackets
- $L = \{ww^R \mid w \in \{0,1\}^*\}$ — even-length palindromes
- Arithmetic expression syntax (as generated by a grammar like $E \rightarrow E+T \mid T$)

12.

A Turing Machine (TM) is a computational model with an unbounded tape (used as both input and working memory) and a read/write head that can move in either direction, combined with a finite control (set of states).

A TM is strictly more powerful than a PDA because its tape provides full random-access read AND write memory that can be revisited any number of times in either direction, whereas a PDA's stack only allows LIFO access to the single most recently pushed symbol. This lets a TM recognize languages a PDA cannot — for example, $L = \{a^n b^n c^n \mid n \geq 1\}$, which requires verifying two independent equality constraints simultaneously, something a single stack cannot do.

13.

A TM is formally a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q0, B, F)$:

- Q — a finite set of states
- Σ — the input alphabet ($\Sigma \subseteq \Gamma$, excludes the blank)
- Γ — the tape alphabet (includes Σ and the blank symbol B)
- δ — the transition function: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (state, symbol read → new state, symbol written, head direction)
- $q0 \in Q$ — the initial state
- $B \in \Gamma$ — the blank symbol, filling all tape cells not yet written
- $F \subseteq Q$ — the set of final/accepting states

14.

Tape: an unbounded sequence of cells, each holding one symbol from Γ; it serves as both the input source and the machine's working memory, and can be read from and written to repeatedly.

Head: a read/write pointer positioned over one tape cell at a time; on each step it reads the symbol under it, and
– per the transition function — writes a (possibly new) symbol back to that cell and then moves one cell Left or Right.

State transitions: $\delta(q, X) = (q', X', D)$ means: while in state q reading symbol X, the machine writes X' in place of X, moves the head in direction D (L or R), and enters state q'. The machine halts (accepting or rejecting) when it reaches a state/symbol combination with no defined transition, or upon reaching a designated halting state.

15.

- Storage in finite control — using the state itself to remember a small, bounded amount of extra information (e.g., the last symbol read), avoiding extra tape passes.

- Multiple tracks — treating each tape cell as holding a tuple of symbols (one per 'track'), simulating several parallel tapes on one physical tape.
- Checking off symbols — marking (overwriting) symbols as they are matched/processed, so repeated passes can find the 'next unprocessed' symbol; used e.g. to compare counts of a's, b's, c's.
- Shifting over — inserting or deleting symbols by shifting an entire block of tape contents one position left or right to make or close a gap.
- Subroutines — a reusable, self-contained block of states (e.g., for subtraction or comparison) that can be 'called' from multiple points in a larger machine and always returns control to the calling context, just like a function in ordinary programming.

16.

It is a design technique where a small, fixed amount of extra information is encoded directly into the TM's current state (e.g., a state literally named [q, X] meaning 'in control state q, remembering that the previously read symbol was X'). This effectively gives the machine a tiny amount of built-in memory beyond the tape, without needing an extra tape pass or extra tracks — useful for tasks like remembering one recently-seen symbol so it can later be compared against another.

17.

Aspect	**FA**	**PDA**	**TM**
Memory	None (finite states)	One stack (LIFO)	Unbounded tape, read/write, both directions
Computational power	Weakest of the three	More powerful than FA	More powerful than PDA
Language class accepted	Regular languages	Context-free languages	Recursively enumerable languages
Cannot accept	$a^n b^n$	$a^n b^n c^n$	(none — limited only by undecidability)

18.

A Deterministic Finite Automaton (DFA) is a finite automaton in which, for every state and every input symbol, there is EXACTLY ONE defined next state (no choices, and no ε-transitions). Formally a 5-tuple $(Q, \Sigma, \delta, q0, F)$, where $\delta: Q \times \Sigma \rightarrow Q$ is a total function. DFAs recognize exactly the regular languages, and every non-deterministic finite automaton (NFA) can be converted into an equivalent DFA (via the subset construction), so DFAs and NFAs have the same expressive power despite the difference in determinism.

19.

A Turing Machine accepts exactly the Recursively Enumerable (RE) languages, also called Type-0 languages in the Chomsky hierarchy — the broadest class of formal languages. This class includes all regular languages and all context-free languages as proper subsets, plus additional languages such as $a^n b^n c^n$ and languages describing general computable problems. (A TM that is also guaranteed to halt on every input, accepting or rejecting, defines the smaller class of Recursive/Decidable languages.)

20.

Although no physical device is a literal Turing Machine (its tape is theoretically unbounded), the model is foundational to computing in several practical ways:

- Theoretical foundation of computer architecture and programming languages — the TM defines the very notion of 'algorithm' and 'computable function' (Church-Turing thesis), underpinning how real CPUs and compilers are designed and reasoned about.

- Compiler design and parsing — TM-related theory (grammars, automata) underlies lexical analysis and syntax parsing in real compilers and interpreters.
- Complexity theory — the TM is the standard reference model used to define and study computational complexity classes such as P and NP, which guide real-world algorithm design and cryptography.
- Decidability/undecidability analysis — TM theory (e.g., the Halting Problem) is used to prove that certain real software-verification tasks (like fully general bug detection) are fundamentally impossible to automate, shaping the design of practical static analysis and verification tools.
- Formal verification and AI — TM-based reasoning underlies formal proofs of program correctness and the theoretical limits of what algorithms (including AI systems) can and cannot compute.